# 099 | 实验手册

## 1. 开始前先写预测

判断下列命题并给理由：分块会减少乘加总量；每块分别softmax后平均等于全局softmax；局部窗口只要足够快就不会丢证据；CPU慢的算法在GPU也必然慢。至少构造一个两块手算反例。

## 2. 运行与测量协议

在本讲目录安装requirements.txt，执行python experiment.py和python tests.py。也可按序运行experiment.ipynb。仅CPU、单线程、float64、d=32、块大小32，长度64/128/256/512。每种算法预热一次，9次正式计时交替顺序。无需网络数据或模型权重。

完整随机输入Q/K/V与两种输出保存在outputs/data.npz；timings.csv保留72次计时。用NumPy重算最大绝对差，不把“完全一致”理解为逐位相同。阅读environment.json确认计时边界。

## 3. 手算在线合并

分数0与log9分成两块，值为0与10。初始m为负无穷、l=a=0；处理第一块后m=0、l=1、a=0；第二块到来后m'=log9、旧缩放r=1/9。计算新的l与a，再求a/l。和分别softmax再平均比较。

接着改变块大小为1、7、32，使用长度33的小数组，检查尾块不满时结果仍一致。注意修改实验应另存输出，避免覆盖已记录的基线。

## 4. 内存账与计时账分开

计算长度512的密集分数矩阵与32乘32分数块各占多少float64字节。列出被排除的Q/K/V、输出、累积量和临时矩阵。不得用这两个数字声称已经测出进程峰值内存或GPU显存。

按method和length汇总9次时间的中位数与四分位间距。解释为什么Python分块代码可能更慢。只报告当前运行环境，不引用别人的GPU速度作为本实验结果。

## 5. 位置检索与能力边界

打开retrieval.csv。逐个长度核对开头、中间、末尾的目标是否位于最近32个位置。全局输出可以用exp(12)/(exp(12)+n−1)独立计算；局部不可见时必须为0。成功阈值预设为0.5，不随结果修改。

设计一个反例：降低目标分数，使大量干扰位置稀释目标权重。说明即使连接存在，也可能检索失败。再解释为何本实验没有位置编码、文本语义和多层学习，因此不能给出真实语言模型的上下文能力结论。

## 6. 提交与验收

提交两张图的独立复算、两块softmax手算、72条计时摘要和12个位置检索结果。6项测试必须通过。报告必须区分前向正确性、真实CPU时间、理论缓冲字节和合成检索结果，并写出未测GPU、未测反向传播两项限制。